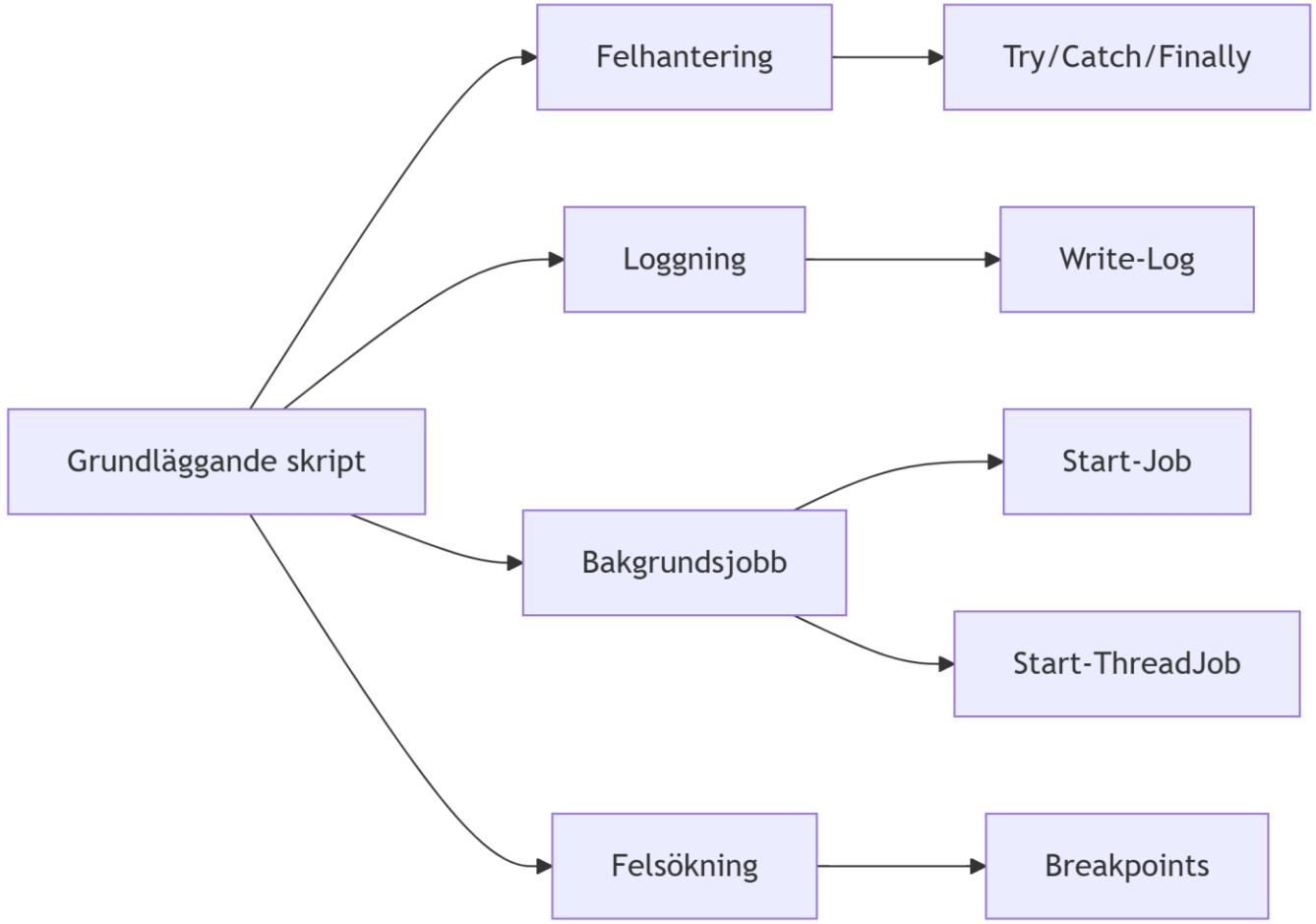
PowerShell Fortsättning

Arseniy Buzyan

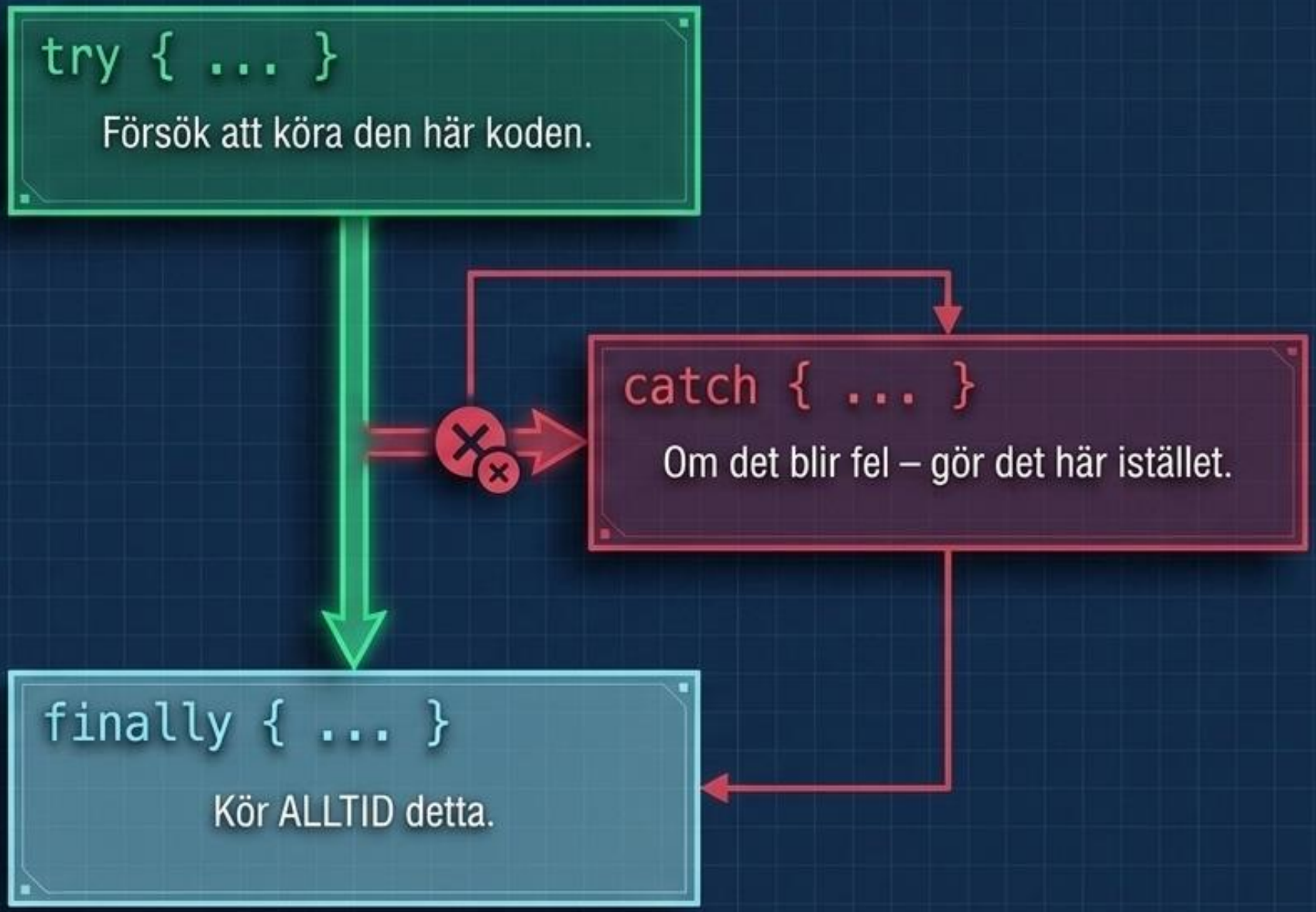
Varför Fortsättning?

- **Nu kan du grunderna – men professionella skript kräver MER:**
- Hantera fel utan att skriptet kraschar
- Logga ALLT som händer
- Köra tunga uppgifter i bakgrunden
- Felsöka smart och effektivt



Varför är detta viktigt? Svenska arbetsgivare vill ha personer som kan skriva STABILA och ROBUSTA PowerShell-skript – inte bara enkla kommandon.

Skyddsnätet: Try, Catch, Finally



⚠ Enligt Microsoft:

finally-blocket körs ALLTID – även om användaren avbryter skriptet med CTRL+C.



Felhantering – Varför Behövs Det?

- Utan felhantering: Ett enda fel = skriptet kraschar!
- Enligt Microsoft: Try/Catch/Finally är den mest kraftfulla metoden för att hantera fel i PowerShell.

Scenario – utan felhantering:

powershell

```
# Detta skript kraschar om filen inte finns!
$innehåll = Get-Content "C:\finnsInte.txt"
Write-Host "Filen lästes!"
```

Scenario – med felhantering:

powershell

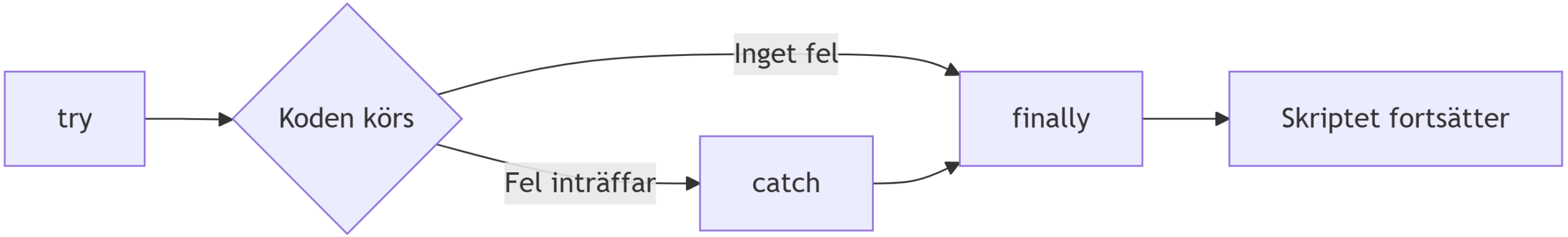
```
try {
    $innehåll = Get-Content "C:\finnsInte.txt"
    Write-Host "Filen lästes!"
} catch {
    Write-Host "❌ Kunde inte läsa filen!"
}
```

Try/Catch/Finally – Grunden i Felhantering



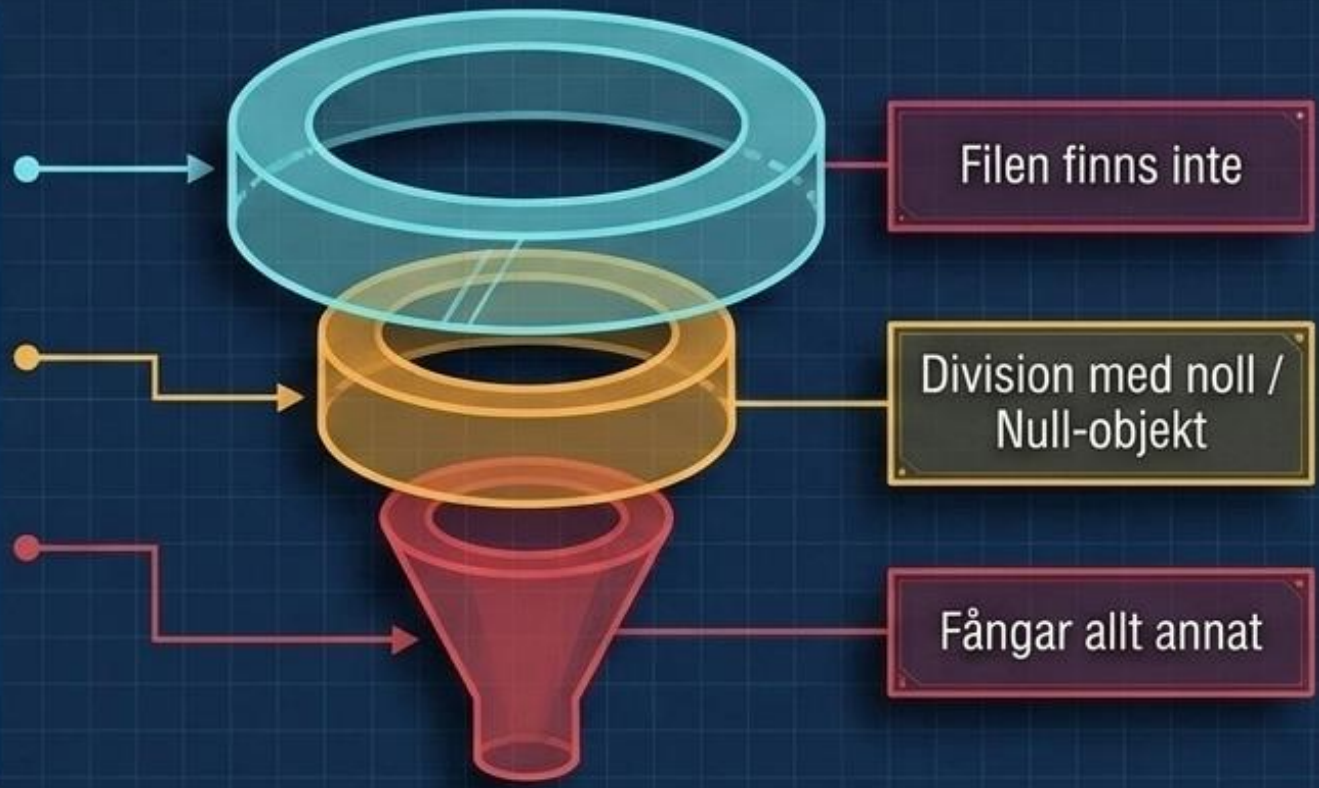
Tre nyckelord som gör dina skript STABILA:

Nyckelord	Vad gör det?
try	Försök att köra den här koden
catch	Om det blir fel – gör det här istället
finally	Kör ALLTID detta (oavsett fel eller inte)



Precisionsfiltrering av feltyper

```
{
  ...
  catch [System.Management.Automation.ItemNotFoundException]
  {
    ...
  }
  catch [System.Management.Automation.RuntimeException]
  {
    ...
  }
  catch
  {
    ...
  }
```



Den första matchande feltypen används.
Olika fel kräver olika åtgärder.

Exempel – med Finally:

Enligt Microsoft: `finally`-blocket körs ALLTID – även om du avbryter skriptet med CTRL+C .

powershell

```powershell
try {
    Get-Content "C:\finnsInte.txt"
} catch {
    Write-Host "❌ Fel: $($_.Exception.Message)"
} finally {
    Write-Host "✅ Detta körs ALLTID – oavsett fel!"
}
```

Fånga Specifika Feltyper

Proffstips: Du kan fånga flera feltyper i samma try-block – den första som matchar används.

Olika fel kräver olika hantering – PowerShell kan fånga specifika feltyper!

Exempel – tre olika feltyper:

```
try {
    # Tre olika sätt att orsaka fel!
    Get-Content "C:\fake_path\fake_file.txt" -ErrorAction Stop
    $null.fakeFunction()        # Metod på null-objekt
    $divideZero = 1/0            # Division med noll
} catch [System.Management.Automation.ItemNotFoundException] {
    Write-Host "❌ Filen hittades inte!"
} catch [System.Management.Automation.RuntimeException] {
    Write-Host "❌ Runtime-fel (t.ex. metod på null)!"
}
```

Feltyp	När inträffar det?
ItemNotFoundException	Filen eller objektet finns inte
RuntimeException	Allmänt fel (t.ex. division med noll, metod på null)
UnauthorizedAccessException	Inte behörighet

Nästlade Try/Catch-block

Du kan lägga try/catch INUTI andra try/catch – för avancerad felhantering.

Varför nästla? Du kan hantera specifika fel i inre catch, och använda yttre catch för att logga alla fel som inte hanterades .

Exempel – inre och yttre try/catch:

powershell

```powershell
try {  # Yttre try/catch
    try {  # Inre try/catch
        Get-Content "C:\fake_path\fake_file.txt" -ErrorAction Stop
    } catch [System.Management.Automation.ItemNotFoundException] {
        Write-Host "Inre catch: Filen hittades inte!"
        # Vi hanterar detta fel här – det bubblar inte upp
    } catch {
        Write-Host "Inre catch: Annat fel – bubblar uppåt!"
        throw  # Skicka felet vidare till yttre catch
    }
} catch {
    Write-Host "Yttre catch: $($_.Exception.Message)"
```

$ErrorActionPreference – Styr Hur PowerShell Hanterar Fel

- Du kan styra hur PowerShell reagerar på fel – globalt eller lokalt.
- Viktigt: PowerShell har två feltyper :
- **Terminating error** = stoppar skriptet direkt
- **Non-terminating error** = visar fel men fortsätter (vanligast)
- $ErrorActionPreference-värden:

Värde	**Vad gör det?**
Continue	Visa fel men fortsätt (STANDARD)
Stop	Alla fel stoppar skriptet
SilentlyContinue	Visa inget fel – fortsätt ändå
Ignore	Ignorera felet helt (loggar inte heller)
Inquire	Fråga användaren vad som ska göras
Break	Gå in i felsökningsläge vid fel

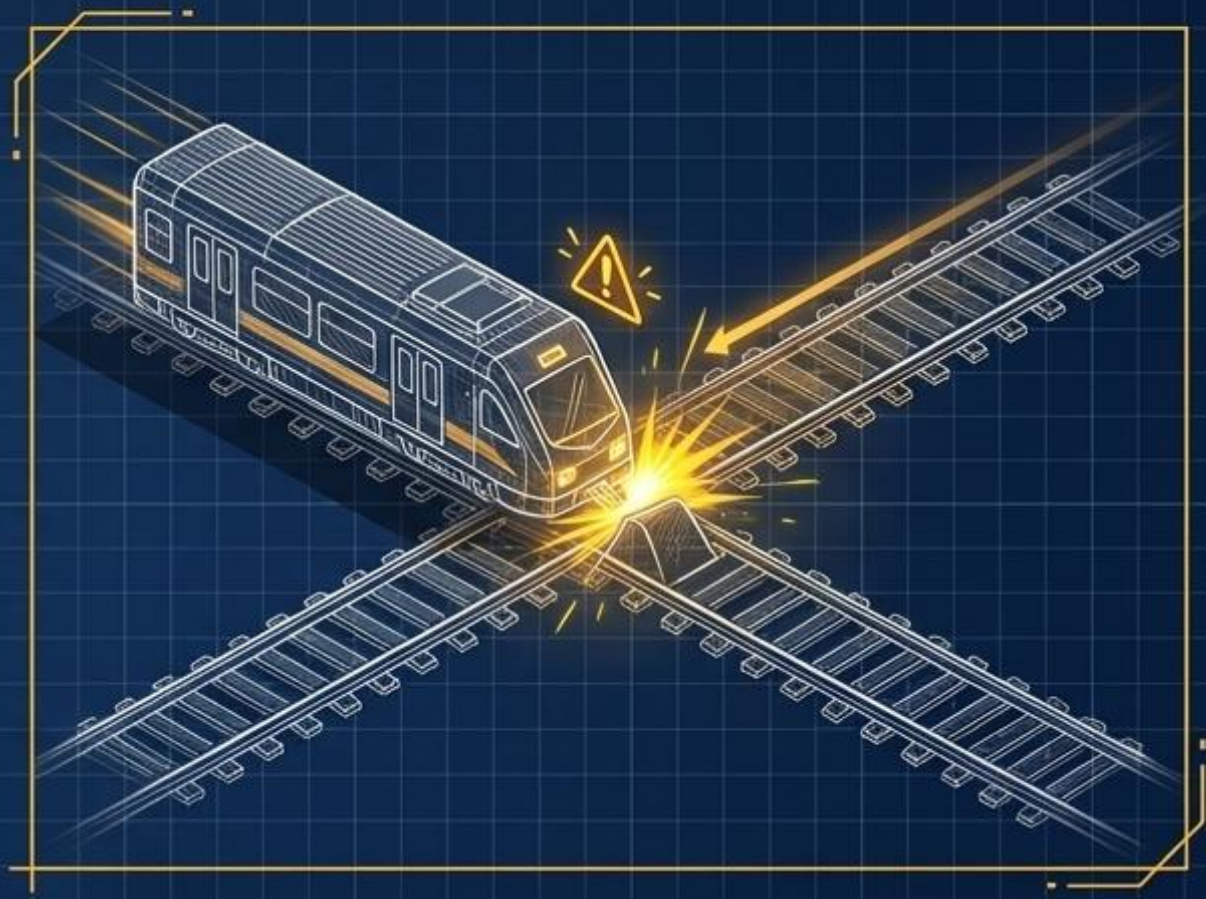


Det osynliga hotet: Två typer av fel



Terminating Error

Stoppar skriptet omedelbart.
Fångas automatiskt av `try/catch`.



Non-terminating Error

Visar felmeddelande i konsolen men *fortsätter* köra (Standardbeteende).
Fångas INTE av en standard `try/catch`!

Exempel – använda Stop:

Proffstips: Använd `-ErrorAction Stop` på specifika kommandon istället för att ändra globalt .

powershell

```powershell
$ErrorActionPreference = "Stop"

try {
    Get-Content "C:\finnsInte.txt"  # Detta stoppar skriptet
    Write-Host "Detta skrivs inte ut!"
} catch {
    Write-Host "❌ Fel upptäcktes: $($_.Exception.Message)"
}
```



$ErrorActionPreference Matrix

`Continue`	Visa fel men **fortsätt** (STANDARD).
`Stop`	Gör om non-terminating till terminating (Stoppar skriptet - bäst för try/catch).
`SilentlyContinue`	Visa inget fel, fortsätt ändå.
`Ignore`	Ignorera felet helt (loggar inte).
`Inquire`	Fråga användaren.
`Break`	Gå in i felsökningsläge.

Varning 1 (Mellanslagsfällan)

`SilentlyContinue` = FEL (kraschar).
`SilentlyContinue` = RÄTT.

Varning 2 (Catch-fällan)

För att fånga ett fel som `Get-ChildItem` måste du lägga till `-ErrorAction Stop` på kommandot, annars ignoreras ditt `catch`-block.

Vanliga Misstag – $ErrorActionPreference

powershell

```powershell
# ❌ FEL – mellanslag!
$ErrorActionPreference = "Silently Continue"  # Fungerar INTE!

# ✅ RÄTT – inget mellanslag!
$ErrorActionPreference = "SilentlyContinue"   # Fungerar!
```

2. Glömma att try/catch bara fångar terminating errors :

powershell

```powershell
# ❌ Fångar INTE felet – Get-ChildItem ger non-terminating error
try {
    Get-ChildItem "C:\finnsInte"
} catch {
    Write-Host "Detta körs ALDRIG!"
}

# ✅ RÄTT – lägg till -ErrorAction Stop
try {
    Get-ChildItem "C:\finnsInte" -ErrorAction Stop
} catch {
    Write-Host "Nu fångas felet!"
}
```

Loggning – Varför och Hur?

- Utan loggning vet du inte vad som hände när skriptet kördes.
- **Varför loggning?**
- Spåra vad som hände
- Felsöka problem i produktion
- Revision och säkerhet
- Veta om skriptet lyckades eller misslyckades
- **Enkel loggningsfunktion:**

```
function Write-Log {
    param(
        [Parameter(Mandatory)]
        [string]$Message,
        [string]$LogPath = "C:\Logs\script.log"
    )
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "$timestamp - $Message"
    Add-Content -Path $LogPath -Value $logEntry
    Write-Host $logEntry
}
```

Anatomin av en loggningsfunktion

Kräver ett meddelande och sätter en standardväg.

```
function Write-Log {
    param(...)
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "$timestamp - $Message"
    Add-Content -Path $LogPath -Value $logEntry
    Write-Host $logEntry
}
```

Konsekvent tidsstämpling är avgörande för felsökning.

Skriver till filen (spårbarhet).

Visar i realtid (användarvänlighet).

Varför logga?

- Spåra händelser, felsöka produktion, revision, och säkerställa status.

Enterprise-Loggning: Nivåer, Färger och Validering

Nivåer & Färger

```powershell
[ValidateSet("INFO", "WARNING", "ERROR", "SUCCESS")]
switch ($Level) {
    "ERROR"   { Write-Host -ForegroundColor Red }
    "SUCCESS" { Write-Host -ForegroundColor Green }
}
```

Defensiv Validering

```powershell
[ValidateScript({Test-Path -Path $_})]
[string]$LogPath = "C:\Logs\app.log"
```

Förhindrar att loggfunktionen själv kraschar om mappen 'C:\Logs' saknas. Körs innan funktionen ens startar.

Proffstips: Använd `Write-Verbose` istället för `Write-Host` för att stödja PowerShells inbyggda verboseloggning.

Avancerad Loggning – Med Nivåer och Färger

Olika loggnivåer gör det lättare att hitta problem.

Proffstips: Använd `Write-Verbose` istället för `Write-Host` för att stödja inbyggd verbose-loggning !

```powershell
function Write-Log {
    param(
        [Parameter(Mandatory)]
        [string]$Message,
        [ValidateSet("INFO", "WARNING", "ERROR", "SUCCESS")]
        [string]$Level = "INFO",
        [string]$LogPath = "C:\Logs\app.log"
    )

    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "[$timestamp] [$Level] $Message"
    Add-Content -Path $LogPath -Value $logEntry

    # Visa med färg i konsolen
    switch ($Level) {
        "ERROR"   { Write-Host -ForegroundColor Red $logEntry }
        "WARNING" { Write-Host -ForegroundColor Yellow $logEntry }
        "SUCCESS" { Write-Host -ForegroundColor Green $logEntry }
        default   { Write-Host $logEntry }
```



Validera Loggsökväg med ValidateScript

Kontrollera att loggmappen finns INNAN du försöker skriva!

Enligt Microsoft: `ValidateScript` kör din scriptblock mot värdet – om den returnerar `$true` godkänns värdet .

powershell

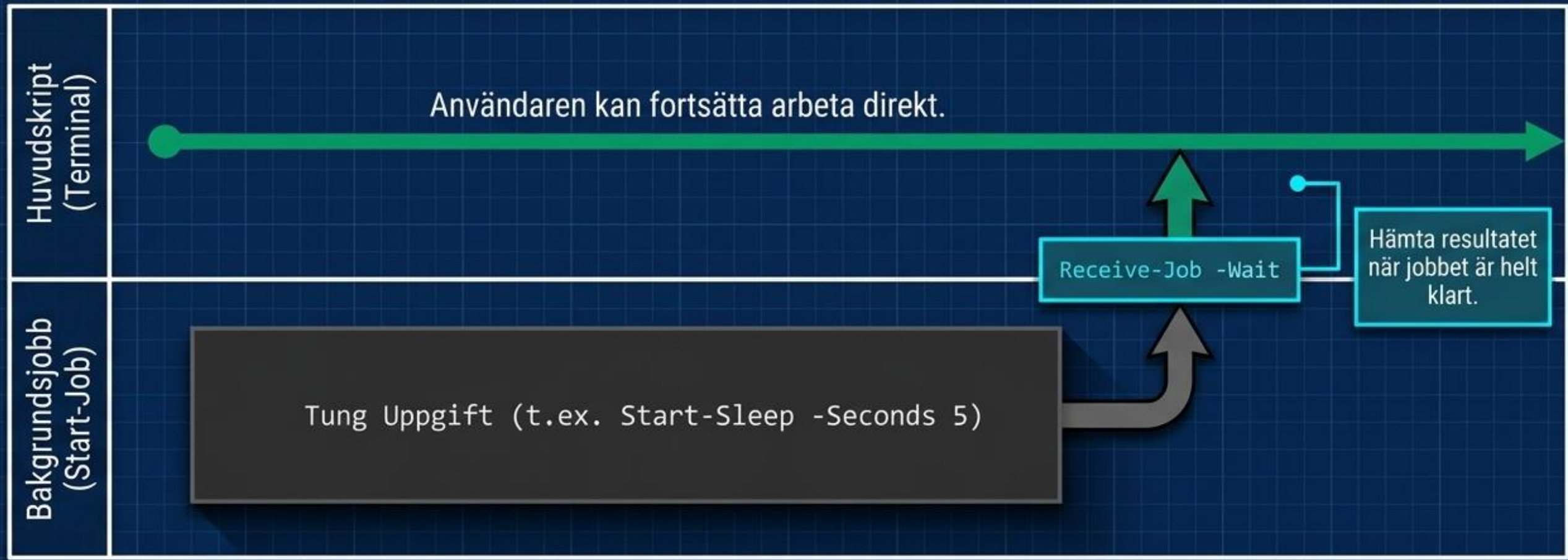
```powershell
function Write-Log {
    param(
        [Parameter(Mandatory)]
        [string]$Message,
        [ValidateScript({Test-Path -Path $_})]
        [string]$LogPath = "C:\Logs\app.log"
    )
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "$timestamp - $Message"
    Add-Content -Path $LogPath -Value $logEntry
}
```

Så här skapar du loggmappen:

powershell

```powershell
$logDir = "C:\Logs"
if (-not (Test-Path $logDir)) {
```

Parallell Exekvering: Huvudtråd vs. Bakgrund



Bakgrundsjobb – Kör Tunga Uppgifter I Bakgrunden

Start-Job låter dig köra tunga uppgifter i bakgrunden medan du fortsätter med annat.



Enkelt exempel:

powershell

```powershell
# Starta ett jobb som kör i bakgrunden
$jobb = Start-Job -ScriptBlock {
    Start-Sleep -Seconds 5
    return "Jobbet är klart!"
}

Write-Host "Väntar på jobb..."  # Detta visas direkt

# Vänta på att jobbet ska bli klart
$resultat = Receive-Job -Job $jobb -Wait
Write-Host $resultat  # Detta visas efter 5 sekunder
```

Proffstips: `Start-Job` startar en separat PowerShell-process för att köra ditt skript i bakgrunden .

Prestandaracet: Job vs. ThreadJob

Kommando	PowerShell Version	Arkitektur	Tid för 20 Jobb (Microsoft Data)
`Start-Job`	PS 5.1+	Startar en helt ny PowerShell-process per jobb	7.58 sekunder
`Start-ThreadJob`	PS 7+	Körs i samma process (trådar)	**2.37 sekunder**

Pro-Tip: Använd `Start-ThreadJob` i PS 7 för massiv prestandaökning, speciellt vid returnering av stora datamängder.

Exempel med Start-ThreadJob

Proffstips: Använd `Start-ThreadJob` i PowerShell 7 för bättre prestanda – särskilt om dina jobb returnerar mycket data .

powershell

```powershell
# PowerShell 7+ krävs!
$jobb1 = Start-ThreadJob -ScriptBlock { Get-Service }
$jobb2 = Start-ThreadJob -ScriptBlock { Get-Process }

# Vänta på båda jobben
$null = $jobb1, $jobb2 | Wait-Job

# Hämta resultaten
$resultat1 = Receive-Job -Job $jobb1
$resultat2 = Receive-Job -Job $jobb2
```



Felsökning – Breakpoints

Breakpoints = "stoppskyltar" där skriptet pausar så att du kan undersöka vad som händer.

Tre typer av breakpoints :

Typ	**När pausar den?**
Line breakpoint	När en specifik rad nås
Variable breakpoint	När en variabel ändras
Command breakpoint	När ett specifikt kommando körs

Breakpoints: Intelligenta Stoppskyltar i Koden



Line Breakpoint

När kod exekverar rad 23.

```
Set-PSBreakPoint -Line 23
```



Variable Breakpoint

När värdet på `$server` ändras.

```
Set-PSBreakPoint -Variable "server"
```



Command Breakpoint

Just innan `Set-ADUser` körs.

```
Set-PSBreakPoint -Command "Set-ADUser"
```

Pausar skriptet så du kan inspektera och manipulera variabler i realtid (F9 i ISE, klicka i marginalen i VS Code).

Så här sätter du breakpoints:

powershell

```powershell
# Line breakpoint – pausa på rad 23
Set-PSBreakPoint -Script "mittskript.ps1" -Line 23

# Variable breakpoint – pausa när $server ändras
Set-PSBreakPoint -Script "mittskript.ps1" -Variable "server"

# Command breakpoint – pausa när Set-ADUser körs
Set-PSBreakPoint -Script "mittskript.ps1" -Command "Set-ADUser"
```

I PowerShell ISE: Högerklicka på raden → "Toggle Breakpoint" (eller F9) .

I VS Code: Klicka i vänstra marginalen vid radnumret för att sätta en breakpoint .

Proffstips: När skriptet pausar kan du undersöka variabler och till och med ändra deras värden innan du fortsätter!

Automatisera med Register-ScheduledJob

Jobben registreras i Windows Task Scheduler. De överlever en omstart av servern och körs helt utan din inblandning.




```
-Once -At (Get-Date).AddMinutes(5)
```

```
-Daily -At "02:00"
```

```
-Weekly -DaysOfWeek "Monday"
```

Schema – Planera Skript Automatiskt

Register-ScheduledJob gör att dina skript körs automatiskt vid bestämda tider.

Exempel – skript som körs om 5 minuter:

powershell

```powershell
# Skapa en trigger (kör om 5 minuter)
$trigger = New-JobTrigger -Once -At (Get-Date).AddMinutes(5)


# Registrera jobbet
Register-ScheduledJob -Name "NattligBackup" -Trigger $trigger -ScriptBlock {
    # Din kod här
    Write-Host "Backup kördes!"
}
```

Vanliga triggers

Kommando	När körs jobbet?
-Once -At (Get-Date).AddMinutes(5)	Om 5 minuter
-Daily -At "02:00"	Varje natt kl 02:00
-Weekly -DaysOfWeek "Monday"	Varje måndag

- Proffstips: Schemalagda jobb registreras i Windows Task Scheduler och finns kvar även efter omstart.

Det Enhetliga Produktionsskriptet

Custom Write-Log Function (Visibility Pillar)

```powershell
function Write-Log {
    param($Message, $Level = "INFO")
    # [Implementation omitted for brevity]
}
```

Safety Pillar

```powershell
try {
    # Code execution starts here

    Write-Log "Startar..."
    Start-ThreadJob -ScriptBlock { ... }
}
```

Initial Log

Performance Pillar: Parallel Execution

Catch Block

```powershell
catch {
    # Error handling logic
    Write-Log -Level ERROR "Ett fel inträffade: $_"
}
```

Error Logging with Color & Timestamp

Cleanup

```powershell
finally {
    # Cleanup operations
    Write-Log "Skriptet avslutas, rensar upp."
    # [Resource cleanup code here]
}
```

Cleanup (Always Runs)

Alla komponenter samverkar. Felet fångas, loggas med färg och tidsstämpel, och systemet rensas upp säkert.

Praktiskt Exempel – Robust Produktionsskript

Proffstips: Använd `$_.ScriptStackTrace` för att se exakt var felet inträffade!

Här är allt vi lärt oss idag i ett enda skript:

powershell

```powershell
# 1. Sätt upp loggning
function Write-Log {
    param(
        [string]$Message,
        [ValidateSet("INFO", "WARNING", "ERROR", "SUCCESS")]
        [string]$Level = "INFO"
    )
    $timestamp = Get-Date -Format "yyyy-MM-dd HH:mm:ss"
    $logEntry = "[$timestamp] [$Level] $Message"
    Add-Content -Path "C:\Logs\app.log" -Value $logEntry
}

# 2. Huvudskript med felhantering
try {
    Write-Log -Message "Skriptet startades" -Level "INFO"

    # Gör något här...
    Write-Log -Message "Åtgärd utförd" -Level "INFO"
```

De 5 Reglerna för Robusta Skript

Pre-Flight Checklist

- [x] **1. Lämna inget åt slumpen:**
  Använd alltid `Try/Catch/Finally` för systemkritiska block.
- [x] **2. Tvinga fram fel:**
  Använd `-ErrorAction Stop` för att fånga non-terminating errors.
- [x] **3. Lämna ett spår:**
  Logga konsekvent med tidsstämplar, nivåer och färger.
- [x] **4. Tänk på prestandan:**
  Använd `Start-ThreadJob` (PS 7+) för tunga eller parallella uppgifter.
- [x] **5. Felsök smart:**
  Sätt breakpoints på variabler och kommandon istället för att gissa med `Write-Host`.

Du skriver inte längre skript. Du bygger system.

Quiz: https://forms.gle/xVPydaHyv3B8SkmZ8